

S E S S I O N 9

Clustering & Dimensionality Reduction

Discovering structure without labels

Today's Roadmap

01	The Unsupervised Shift	15 min
02	K-Means Clustering	40 min
03	Hierarchical Clustering	25 min
04	DBSCAN	20 min
05	PCA	35 min
06	t-SNE	15 min
07	Putting It Together	20 min

The Unsupervised Shift

WHY

Supervised

We have labels y

Goal: predict y from X

Sessions 1–8



Unsupervised

No labels — only X

Goal: discover structure

Today's session

Business examples: customer segmentation, anomaly detection, data compression, recommendation engines, visualization of high-dimensional data

The Curse of Dimensionality

WHY

As dimensions increase:

- Distances between points become nearly equal
- Data becomes sparse — neighborhoods unreliable
- Noise accumulates across features
- Models need exponentially more data

Concrete example

10 points in 1D: well-covered

10 points in 2D: sparse grid

10 points in 10D: each point

is essentially alone

Need 10^{10} points to cover

10D as densely as 1D!

This is why we need dimensionality reduction: fewer dimensions → more meaningful distances, less noise, better clustering.

C A S E S T U D Y

CustomerScope

A retail company wants to segment its 10,000 customers for targeted marketing — but has no predefined groups.

Features available:

- Annual spending
- Visit frequency
- Avg basket size
- Product categories
- Recency of last visit
- Online vs in-store ratio

K-Means Clustering

WHY

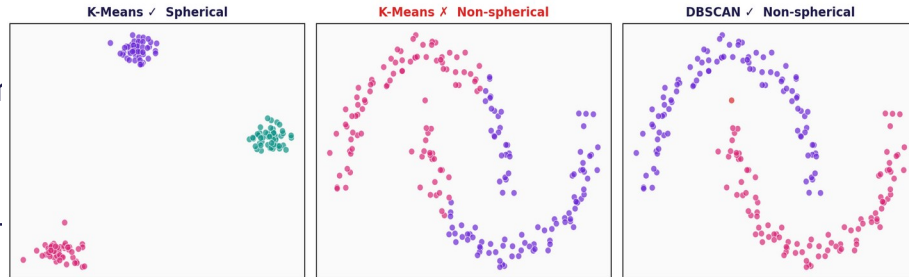
"Group similar customers without predefined categories"

You have 10,000 customers and 6 features.

Marketing wants 3-5 actionable segments.

K-Means is the simplest, most widely used clustering algorithm.

It partitions data into K groups by minimizing within-distances.

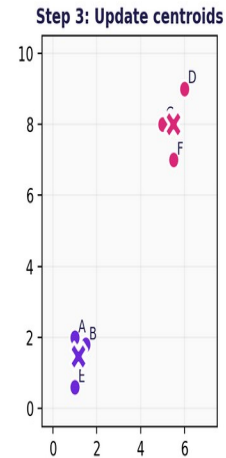
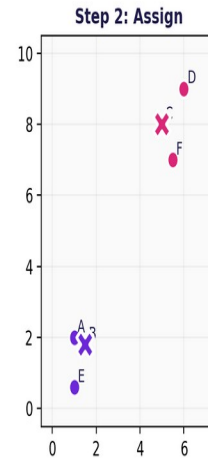
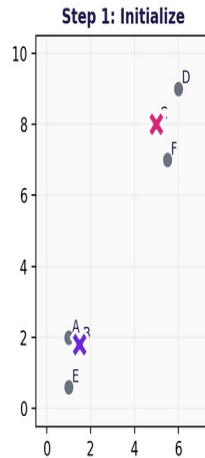


Key assumption: K-Means assumes spherical, equally-sized clusters. We'll see alternatives for other shapes.

K-Means: The Algorithm

WHAT

- 1 Choose K (number of clusters)
- 2 Initialize K centroids (randomly or k-means++)
- 3 Assign each point to its nearest centroid
- 4 Recompute centroids as cluster means
- 5 Repeat steps 3-4 until convergence



K-Means iteration: initialize → assign → update centroids

Convergence = assignments no longer change (or WCSS change < threshold).

K-Means: Worked Example

WHAT

6 points, $K = 2$. Initial centroids: $\mu_1 = B(1.5, 1.8)$, $\mu_2 = C(5, 8)$

Point	Coords	$d(\mu_1)$	$d(\mu_2)$	Assign
A	(1, 2)	0.54	7.21	C_1
B	(1.5, 1.8)	0.00	7.14	C_1
C	(5, 8)	7.14	0.00	C_2
D	(6, 9)	8.35	1.41	C_2
E	(1, 0.6)	1.28	8.37	C_1
F	(5.5, 7)	6.49	1.12	C_2

New centroids:

$$\mu_1 = \text{mean}(A, B, E) = (1.17, 1.47)$$

$$\mu_2 = \text{mean}(C, D, F) = (5.50, 8.00)$$

$$\begin{aligned} \text{WCSS} &= \sum ||\mathbf{x}_i - \mu_k||^2 \\ &= (0.54^2 + 0^2 + 1.28^2) \\ &\quad + (0^2 + 1.41^2 + 1.12^2) = 5.17 \end{aligned}$$

Repeat assign $\rightarrow$ update until centroids stop moving. Here, convergence happens after iteration 2.

K-Means: Objective Function

WHAT

$$J = \sum_{k=1..K} \sum_{(x_i \in C_k)} \|x_i - \mu_k\|^2$$

J

Total within-cluster sum of squares (WCSS) — lower is better

K

Number of clusters (hyperparameter you choose)

C_k

Set of points assigned to cluster k

μ_k

Centroid (mean) of cluster k : $\mu_k = (1/|C_k|) \sum x_i$

$\|x_i - \mu_k\|_2$

Squared Euclidean distance from point to its centroid

K-Means is an iterative optimization: it minimizes J , but may find a local minimum, not the global one. → Use multiple restarts (n_{init}).

K-Means++: Smart Initialization

WHAT

Random initialization can lead to poor convergence. K-Means++ spreads initial centroids apart:

- 1 Pick the first centroid μ_1 uniformly at random from the data
- 2 For each point x , compute $D(x)$ = distance to nearest existing centroid
- 3 Pick next centroid with probability proportional to $D(x)^2$
- 4 Repeat steps 2–3 until K centroids are chosen

Intuition

Points far from existing centroids have higher probability of being chosen as the next centroid.

This ensures centroids are spread across the data, avoiding the 'all centroids in one cluster' trap.

sklearn default: `init='k-means++'`. Always use it. Random init is almost never better.

K-Means: Implementation

HOW

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Always scale before clustering!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=4, init='k-means++',
                 n_init=10, random_state=42)
labels = kmeans.fit_predict(X_scaled)

# Access results
centroids = kmeans.cluster_centers_
inertia = kmeans.inertia_ # WCSS value
```

Key Parameters

n_clusters

K — number of clusters

init

'k-means++' (default, recommended)

n_init

Number of restarts (default 10)

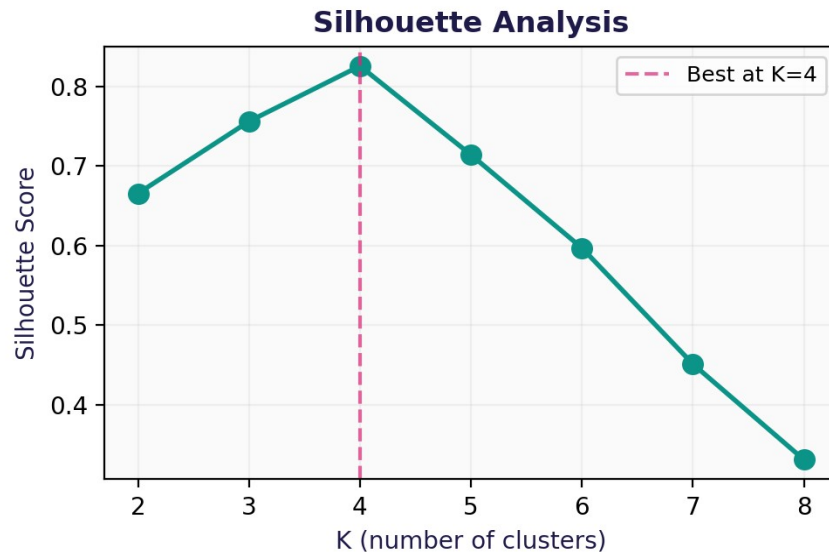
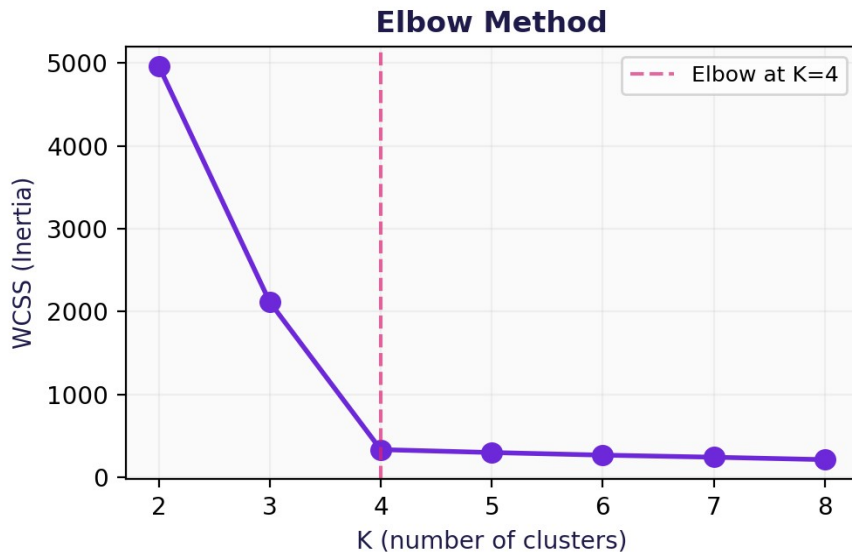
random_state

For reproducibility

Always StandardScaler before K-Means. Features on different scales distort Euclidean distance.

Choosing K

WHAT



Silhouette score: $s(i) = (b - a) / \max(a, b)$ where **a** = mean distance to points in same cluster, **b** = mean distance to nearest other cluster. Range $[-1, +1]$. Higher = better.

Choosing K: Implementation

HOW

```
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
```

```
# Elbow method
wcss = []
K_range = range(2, 10)
for k in K_range:
    km = KMeans(n_clusters=k, n_init=10,
                random_state=42)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
```

```
plt.plot(K_range, wcss, 'o-')
plt.xlabel('K'); plt.ylabel('WCSS')
plt.title('Elbow Method'); plt.show()
```

```
# Silhouette analysis
for k in range(2, 8):
    km = KMeans(n_clusters=k, n_init=10,
                random_state=42)
    labels = km.fit_predict(X_scaled)
    score = silhouette_score(X_scaled,
                             labels)
    print(f"K={k}:
silhouette={score:.3f}")
```

```
# Pick K with highest silhouette score
# (also consider business requirements)
```

Neither method gives a definitive answer. Always combine with domain knowledge: does K=4 produce segments marketing can actually act on?

K-Means: Pitfalls

Assumes spherical clusters

Fails on elongated or ring-shaped groups

Sensitive to initialization

Use k-means++ and multiple restarts (n_init)

Must choose K in advance

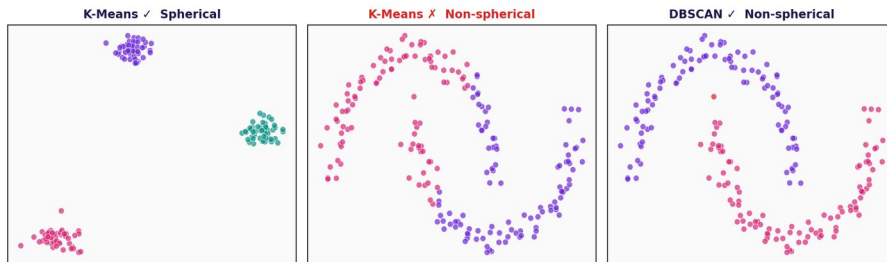
Elbow/silhouette help but are heuristics

Sensitive to scale

Always standardize features first

Outliers distort centroids

Consider removing outliers or using K-Medoids



K-Means fails on non-spherical shapes — DBSCAN handles them

K-Means is a starting point, not the final answer. Always try at least one alternative.

Distance Metrics

WHAT

Clustering results depend entirely on how you measure 'closeness':

Metric	Formula	Best for	Sensitive to scale?
Euclidean	$\sqrt{\sum (x_i - y_i)^2}$	Dense, spherical clusters	Yes — always scale
Manhattan	$\sum x_i - y_i $	Sparse / high-dim data	Yes — always scale
Cosine	$1 - (x \cdot y) / (\ x\ \ y\)$	Text / direction matters	No (normalized)

K-Means always uses Euclidean distance

(because the centroid update formula assumes it). For other metrics, use K-Medoids.

Rule of thumb: if features are numeric with different units → Euclidean + StandardScaler. If features are counts or frequencies → consider Cosine.

Hierarchical Clustering

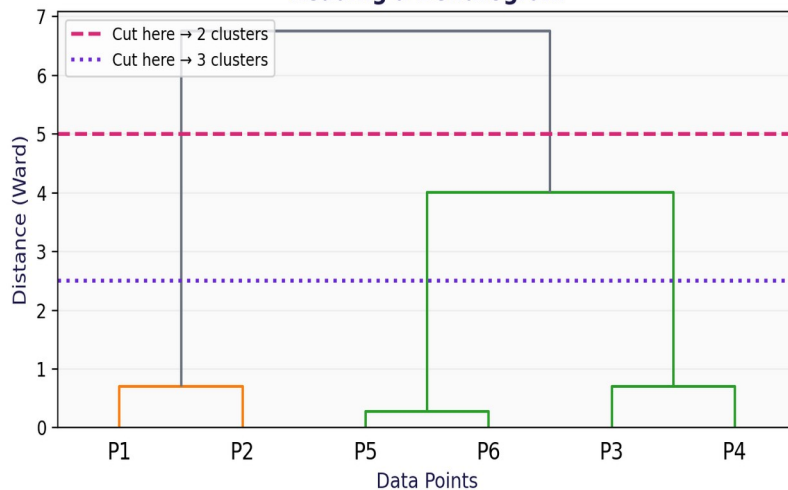
WHY

When you want to explore cluster structure at multiple levels — without choosing K upfront

Agglomerative (bottom-up) algorithm:

- 1 Start: each point is its own cluster
-
- 2 Merge the two closest clusters
-
- 3 Repeat until one cluster remains
-
- 4 Cut the dendrogram at desired level → your clusters
-

Reading a Dendrogram



Unlike K-Means, hierarchical clustering produces a full hierarchy. You choose the number of clusters after seeing the structure.

Linkage Methods

WHAT

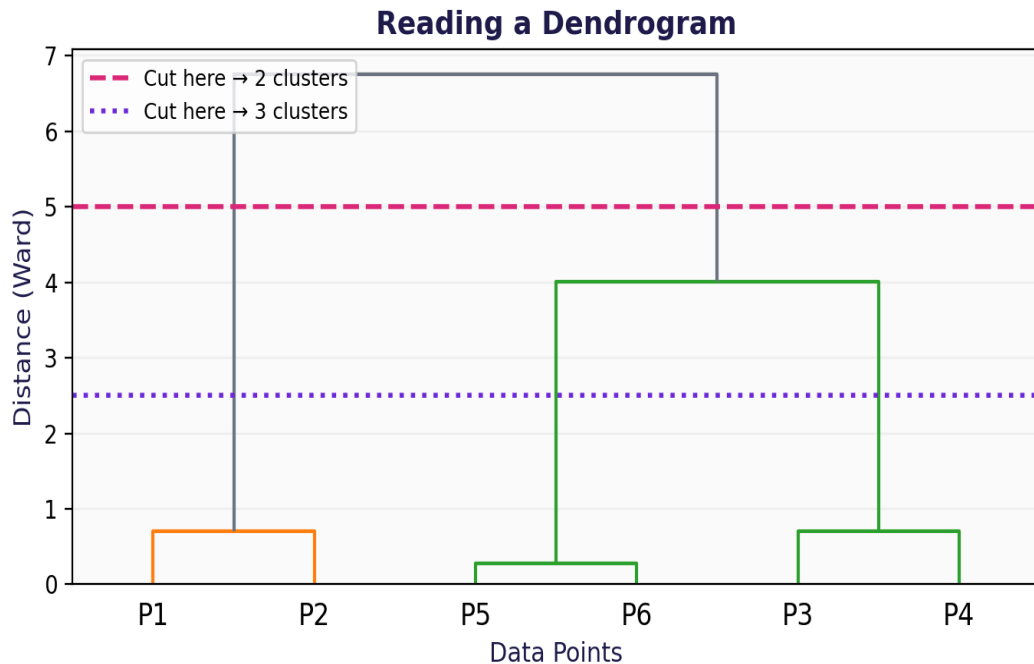
"Distance between two clusters" can mean different things:

Linkage	Measures	Tendency
Single	Min distance between any pair	Chains (elongated)
Complete	Max distance between any pair	Compact, equal-size
Average	Mean pairwise distance	Balanced
Ward	Increase in total WCSS after merge	Compact, similar to K-Means

Recommendation: start with Ward linkage — it produces compact clusters similar to K-Means and is the most commonly used in practice.

Reading a Dendrogram

WHAT



How to read:

Y-axis

= merge distance. Higher = more dissimilar clusters being merged.

Horizontal line

= a merge event. The two branches below it are the clusters that merged.

Cut horizontally

to get clusters. Cut at height 5 → 2 clusters. Cut at 2.5 → 3 clusters.

Tall vertical gaps

= natural cluster boundaries. Look for the biggest jump in merge distance.

The dendrogram is a decision tool: it shows you what K values are natural for your data.

Hierarchical Clustering: Code

HOW

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram,
linkage
import matplotlib.pyplot as plt

# Fit clustering
hc = AgglomerativeClustering(n_clusters=4,
                             linkage='ward')
labels = hc.fit_predict(X_scaled)

# Visualize dendrogram (uses scipy, not sklearn)
Z = linkage(X_scaled, method='ward')
dendrogram(Z, truncate_mode='level', p=5)
plt.xlabel('Samples')
plt.ylabel('Distance')
plt.title('Dendrogram')
plt.show()
```

Tips

truncate_mode='level'

limits depth of the dendrogram display
(useful for large datasets)

linkage='ward'

good default — minimizes variance within
clusters

n_clusters=None + distance_threshold

cut the dendrogram at a specific distance
instead of specifying K

Scalability:

$O(N^2)$ memory, $O(N^3)$ time — avoid for $N > 10,000$

DBSCAN

Density-Based Spatial Clustering of Applications with Noise

WHAT

Two parameters:

eps (ϵ)

radius of neighborhood

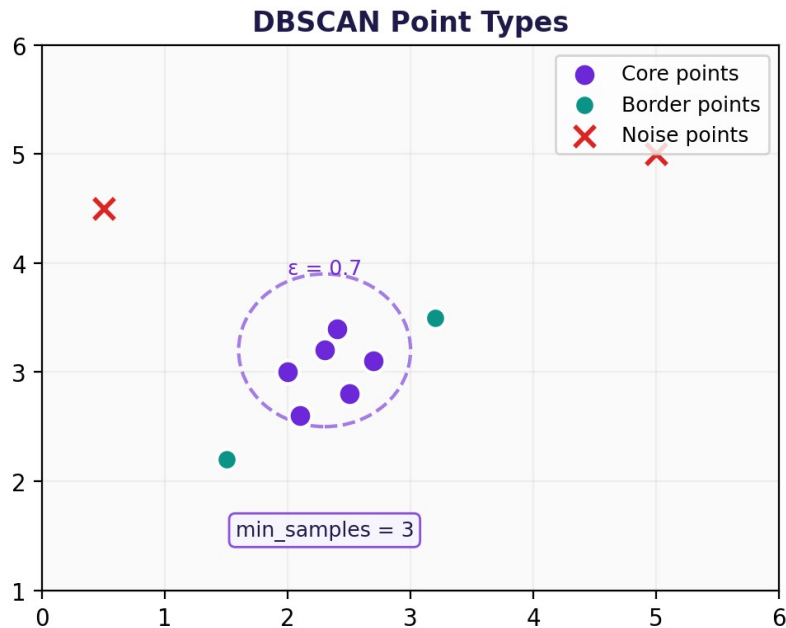
min_samples

minimum points to form a dense region

Three types of points:

-  **Core point** $\geq \text{min_samples}$ within ϵ
-  **Border point** Within ϵ of core, $< \text{min_samples}$ neighbors
-  **Noise point** Neither core nor border $\rightarrow$ outlier (label = -1)

Key advantage: DBSCAN doesn't need K, discovers arbitrary-shaped clusters, and identifies outliers natively.

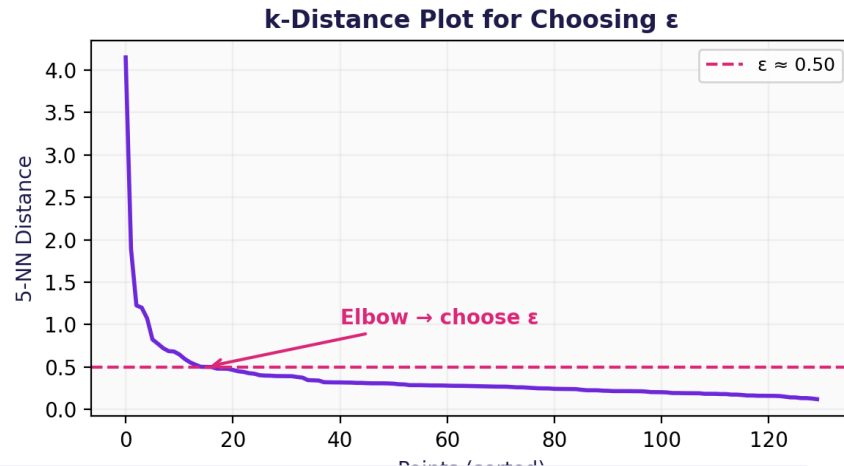


DBSCAN: Choosing ϵ

WHAT

The k-distance plot is the standard method for selecting ϵ :

1. For each point, compute distance to its k-th nearest neighbor
($k = \text{min_samples}$)
2. Sort distances in decreasing order and plot them
3. Look for the 'elbow' — the sharp bend where noise transitions to clusters
4. The y-value at the elbow $\approx$ your ϵ



```
from sklearn.neighbors import NearestNeighbors
nn = NearestNeighbors(n_neighbors=5)
nn.fit(X_scaled)
distances, _ = nn.kneighbors(X_scaled)
k_dist = np.sort(distances[:, 4])[:, -1]
plt.plot(k_dist)
plt.ylabel('5-NN Distance'); plt.show()
```

DBSCAN: Implementation

HOW

```
from sklearn.cluster import DBSCAN

db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X_scaled)

n_clusters = len(set(labels)) - (1 if -1 in labels
else 0)
n_noise = list(labels).count(-1)

print(f"Clusters: {n_clusters}, Noise: {n_noise}")
```

Rules of thumb

$\text{min_samples} \geq D + 1$

(D = number of dimensions)

eps: use k-distance plot

(see previous slide)

HDBSCAN:

auto-selects ϵ , handles varying density. Try it next!

Strengths	Limitations
No need to choose K	Sensitive to ϵ and min_samples
Detects arbitrary cluster shapes	Struggles with varying density
Identifies outliers (label = -1)	Less effective in very high dimensions
No assumption of cluster shape	Cannot handle clusters of very different densities

Clustering Methods: When to Use What

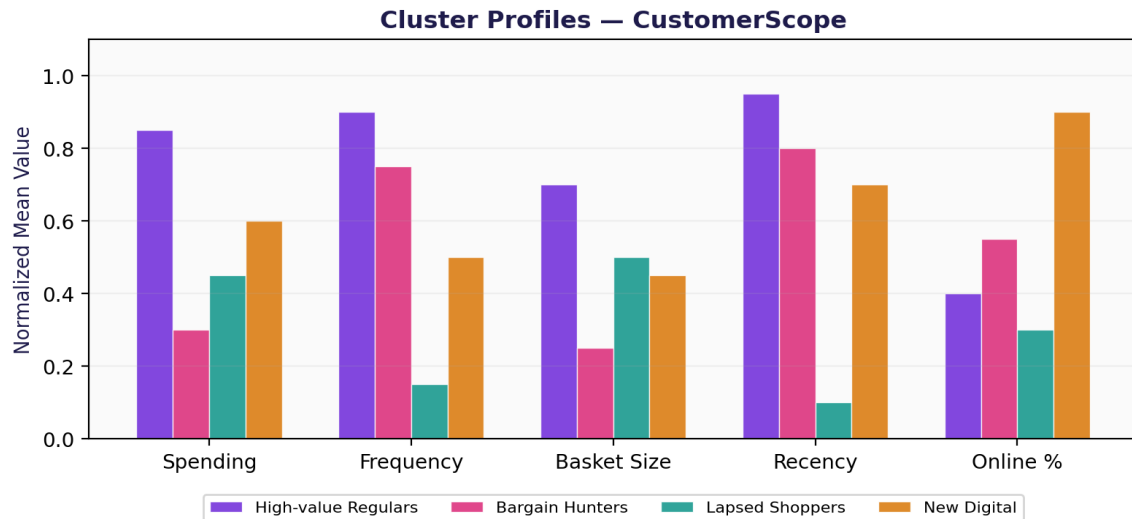
	K-Means	Hierarchical	DBSCAN
Choose K?	Yes (required)	Optional (cut dendrogram)	No (auto-detected)
Cluster shape	Spherical only	Depends on linkage	Arbitrary
Outlier handling	Poor	Poor	Built-in (label = -1)
Scalability	Fast: $O(NKt)$	Slow: $O(N^2)$ to $O(N^3)$	Moderate: $O(N \log N)$
Interpretability	Centroids	Dendrogram	Density regions
Best for	Equal-sized, round groups	Exploring hierarchy	Irregular shapes + noise

Decision framework: start with K-Means (fast baseline), check dendrogram for hierarchy, try DBSCAN if shapes are non-spherical or you suspect outliers.

Cluster Profiling

HOW

After clustering: what do these groups actually mean?



Profiling steps:

1. Compute mean feature values per cluster
2. Normalize to compare features on different scales
3. Visualize with bar charts or radar plots
4. Name each cluster based on dominant traits
5. Validate with domain experts

This is the business-critical step. Clusters that can't be named and acted upon are useless — even if the silhouette score is high.

Principal Component Analysis

WHY

Reduce dimensions while preserving maximum information.

Visualization

Project 50 features onto 2D for human understanding

Noise reduction

Drop low-variance components that mostly capture noise

Speed

Fewer features → faster downstream models (KNN, SVM)

Multicollinearity

Correlated features? PCA creates uncorrelated components

PCA is not feature selection — it creates new features (linear combinations of all originals). You lose individual feature interpretability.

PCA: The Mathematics

WHAT

Covariance Matrix

$$S = (1/N) \sum_n (x_n - \bar{x})(x_n - \bar{x})^T$$

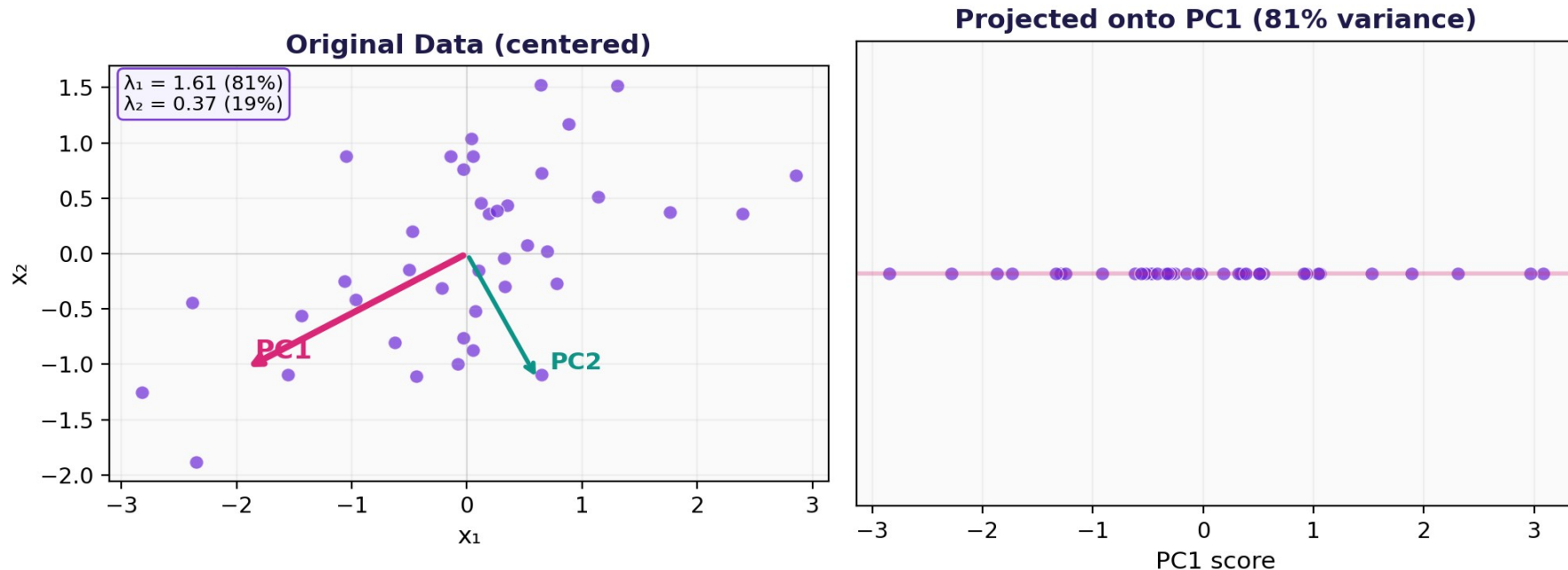
Eigendecomposition

$$S \cdot v = \lambda \cdot v$$

- S** Covariance matrix (symmetric, positive semi-definite)
- v** Eigenvectors → principal component directions (orthogonal)
- λ** Eigenvalues → variance captured by each direction
- $\bar{x}$** Mean of the data — subtract it to center the data first

PCA: Worked Example

WHAT



PC1 captures 81% of variance → projecting onto PC1 alone preserves most structure. PC2 adds 19% more — keeping both retains 100%.

PCA: Step by Step

- 1 Standardize the data (zero mean, unit variance)
- 2 Compute the covariance matrix S
- 3 Compute eigenvectors and eigenvalues of S
- 4 Sort eigenvectors by decreasing eigenvalue
- 5 Keep top k eigenvectors (your principal components)
- 6 Project data onto the k -dimensional subspace

Choosing k Scree plot:

Plot eigenvalues, look for the 'elbow'
(similar to K-Means!)

Cumulative variance:

Keep k components that explain $\geq 90\text{--}95\%$
of total variance

Example:

$\lambda = [4.5, 2.1, 0.3, 0.1]$

Cumulative: 64%, 94%, 98%, 100%

→ $k = 2$ captures 94%

Always standardize before PCA! Without it, the feature with the largest scale dominates the first component.

PCA: Implementation

HOW

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Step 1: Standardize
X_scaled = StandardScaler().fit_transform(X)

# Step 2: Fit PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Step 3: Explained variance
print(pca.explained_variance_ratio_)
# e.g. [0.45, 0.28] → 73% in 2 components

# Step 4: Component loadings (NEW!)
```

Loadings bridge interpretability and math: they explain what each PC captures in business terms.

```
pca.components_,
columns=['PC1', 'PC2'],
index=feature_names
)
```

Component Loadings

`pca.components_`

tells you what each PC 'means':

Example loadings for PC1:

Spending:	0.62
Frequency:	0.55
Basket:	0.48
Online:	-0.12

→ PC1 ≈ 'overall engagement'

PCA: What It Is Not

X PCA selects the best features

→ PCA creates new features — linear combinations of all originals

X High variance = useful

→ Maximum variance direction may not be the most informative for your task

X PCA works on any structure

→ PCA is linear — it cannot capture nonlinear relationships (use kernel PCA or t-SNE)

X PCA components are interpretable

→ Components are mathematical blends — look at loadings to interpret them

X Skip standardization

→ Without scaling, high-variance features dominate PCA entirely

t-SNE: Nonlinear Visualization

WHAT

PCA preserves global variance. But what about local structure?

Core idea: if two points are close in high-dimensional space, they should remain close in 2D. Preserves local neighborhoods.

Parameters

- perplexity controls neighborhood size (try 5–50)
- Results vary with random seed — run multiple times

Warnings

- Inter-cluster distances on the plot may be unreliable
- Cluster sizes on the plot can be misleading
- Visualization only never use as a preprocessing step

Also worth knowing: UMAP — faster alternative, often better global structure preservation. Worth trying as a drop-in replacement.

	PCA	t-SNE
Type	Linear	Nonlinear
Preserves	Global variance	Local neighborhoods

t-SNE: Implementation

HOW

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Fit t-SNE (always on scaled data)
tsne = TSNE(n_components=2, perplexity=30,
            random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# Color by cluster labels from K-Means
plt.scatter(X_tsne[:, 0], X_tsne[:, 1],
            c=kmeans_labels, cmap='viridis',
            alpha=0.6, s=20)
plt.title('t-SNE visualization')
plt.xlabel('t-SNE 1'); plt.ylabel('t-SNE 2')
plt.colorbar(label='Cluster')
plt.show()
```

Practical tips

Apply PCA first

if you have many features (>50). Reduces noise and speeds up t-SNE.

Try different perplexity

values (5, 15, 30, 50) — results change significantly.

Run multiple seeds

to check if structure is consistent.

UMAP alternative:

`pip install umap-learn`

t-SNE is non-parametric: you can't transform new test data. Fit on training only, or use PCA for production pipelines.

A Critical Warning

Clustering will always return groups
— even if no natural clusters exist.

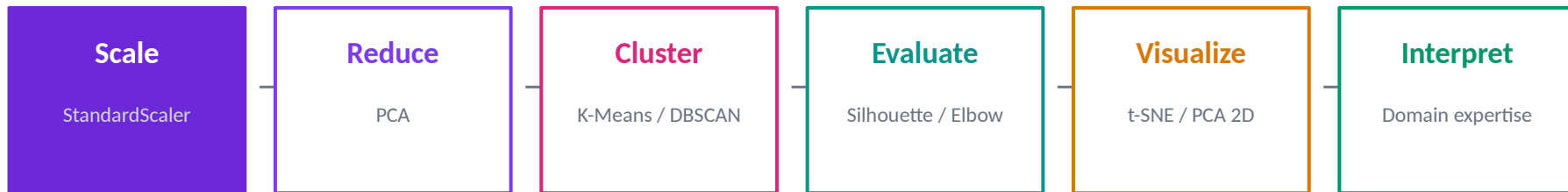
Run K-Means on completely random, uniform data.

It will happily produce K clusters. They mean nothing.

A pretty 2D scatter plot is not proof of meaningful structure.

Always validate: domain expertise + evaluation metrics + stability checks

The Unsupervised Pipeline



CustomerScope — Full Pipeline

1. Scale 6 features → StandardScaler
2. PCA → 3 components capture 92% variance
3. K-Means (K=4) → 4 customer segments
4. Silhouette = 0.58 → decent separation
5. t-SNE scatter confirms visual grouping
6. Profile clusters: 'High-value regulars', 'Bargain hunters', 'Lapsed shoppers', 'New digital'

Key Takeaways

- Unsupervised learning discovers structure without labels
- K-Means is simple but assumes spherical clusters — always try alternatives
- DBSCAN handles arbitrary shapes and identifies outliers natively
- Hierarchical clustering reveals multi-level structure via dendrograms
- PCA reduces dimensions linearly — always standardize first
- t-SNE is for visualization only — don't trust distances on the plot
- Clustering always produces groups — validate with metrics and domain knowledge
- The profiling step turns math into business insight